Progress

When to use this

Choose the progress APIs when you need to show the advancement of long-running tasks such as downloads, migrations, or processing queues. Consoul's progress bar works well in environments where full-screen UIs are impractical.

Key types

- `ProgressBar` – Tracks total units and renders textual progress bars.
- `FixedMessage` – Underlies the progress bar layout by reserving buffer positions.
- `RenderOptions` – Provides default colours for completed and remaining segments.

Minimal example

```
var bar = new ProgressBar("Synchronising data")
{
    BarWidth = Math.Min(Console.BufferWidth - 10, 80),
    BlockCharacter = '#'
};

for (int i = 0; i <= 10; i++)
{
    double progress = i / 10d;
    bar.Update(progress, message: $"Processed {i * 100} items");
    Thread.Sleep(150);
}
```

Advanced tips

- **Fractional updates** – The `Update(double progress, …)` overload expects a 0–1 value; clamp your input to avoid rendering glitches. Pair the fraction with a message to surface ETA or processed counts, and round your value before displaying to avoid jitter from floating-point noise.
- **Minimal width & resizing** – Guard against extremely small console windows by clamping `BarWidth` to a sensible minimum (e.g., 20 characters). Recalculate the width when handling `Consoul.WindowResized` to keep bars aligned, and persist the calculated width on the instance so every subsequent refresh stays consistent.
- **Color overrides & glyphs** – Pass explicit colours into `Update` for special states (warning, error) without changing global render options. Swap `BlockCharacter` to reflect success/failure with ASCII glyphs such as `#`, `=`, or , and align your colour choices with `RenderOptions.DefaultScheme` so the bar respects your broader theme.

- **Integration with tasks** – Combine progress bars with views to create dashboards that refresh status in response to user input. `ProgressBar` can be fed by background `Task` callbacks—just coordinate access if multiple threads update the same instance and throttle updates to avoid flicker on high-frequency jobs.
- **Multiple bars** – Instantiate multiple `ProgressBar` instances with different labels to track concurrent operations. Render them sequentially and refresh in-place by reusing each bar instance instead of recreating them for every update; consider grouping related bars with `Consoul.Write` headings for readability.
- **Completion hooks** – Listen for task completion and call `Update(1, message: "Completed")` to signal that the bar is finished, then optionally render a summary line and switch the bar's colour scheme to a success or failure tone.